



DATABASE_ARCHITECTURE.md

Project: CommercePilot

Version: 1.0

Architecture Style: Multi-Tenant Relational Design (PostgreSQL)

ORM: Prisma

Database: PostgreSQL (Supabase)

1. Database Design Philosophy

CommercePilot database is designed with these principles:

1. Multi-Tenant First

Every record belongs to a tenant (business owner).

No exception.

2. Auditability by Default

Every important action must be traceable.

3. AI-Friendly Structure

Data must be structured so AI can reliably:

- extract
- interpret
- reprocess
- explain

4. No Cross-Tenant Leakage

Tenant isolation is enforced at schema level + application layer.

2. Multi-Tenant Strategy

We use **Shared Database + Tenant ID Isolation**

Core Rule:

Every table includes:

- `tenant_id` (UUID)

Why this model?

- ✓ Simple to implement
 - ✓ Scalable for MVP → enterprise transition
 - ✓ Works perfectly with Supabase + Prisma
 - ✓ Easy for examiners to understand
-

3. Global Base Fields (All Tables)

Every main table MUST include:

`id`: UUID (Primary Key)
`tenant_id`: UUID (Indexed)
`created_at`: timestamp
`updated_at`: timestamp
`deleted_at`: timestamp (nullable soft delete)

Rules:

- `id` = UUID v4
 - `created_at` = auto timestamp
 - `updated_at` = auto update trigger
 - `deleted_at` = soft delete only (NEVER hard delete in business tables)
-

4. Core Database Modules

4.1 USERS

Represents system users (owners, staff)

```
User {  
  id  
  tenant_id  
  name  
  email  
  password_hash  
  role (OWNER | STAFF | ADMIN)  
  is_active  
  last_login  
}
```

4.2 CUSTOMERS

Customers coming from WhatsApp / orders

```
Customer {  
  id  
  tenant_id  
  name  
  phone  
  email  
  address  
  total_orders  
  lifetime_value  
}
```

4.3 PRODUCTS

Product catalog used by AI (RAG source)

```
Product {  
  id  
  tenant_id  
  name  
  description  
  sku  
  price
```

```
stock_quantity
attributes (JSONB)
is_active
}
```

Important:

- AI ONLY searches this table for product matching
 - This is the **single source of truth for RAG**
-

4.4 ORDERS

Core business entity

```
Order {
  id
  tenant_id
  customer_id
  status (
    PENDING_AI,
    WAITING_APPROVAL,
    APPROVED,
    REJECTED,
    SYNCED
  )
  total_amount
  ai_confidence_score
  source (WHATSAPP | MANUAL | API)
  delivery_address
  notes
}
```

4.5 ORDER_ITEMS

```
OrderItem {
  id
  tenant_id
  order_id
  product_id
  quantity
  unit_price
}
```

```
    subtotal
  }
```

4.6 WHATSAPP_MESSAGES

Stores raw conversation data

```
WhatsAppMessage {
  id
  tenant_id
  customer_id
  message_text
  direction (INBOUND | OUTBOUND)
  message_type (TEXT | IMAGE | AUDIO)
  timestamp
  ai_processed (boolean)
}
```

4.7 AI_PROCESSING_LOGS

Critical for debugging AI decisions

```
AIProcessingLog {
  id
  tenant_id
  message_id
  stage (
    INTENT_DETECTION,
    PRODUCT_EXTRACTION,
    ORDER_GENERATION,
    CONFIDENCE_SCORE
  )
  input_data (JSONB)
  output_data (JSONB)
  model_used
  processing_time_ms
}
```

4.8 AI_ORDERS (DRAFTS)

Intermediate AI-generated orders BEFORE approval

```
AIDraftOrder {  
  id  
  tenant_id  
  customer_id  
  extracted_text  
  structured_data (JSONB)  
  confidence_score  
  status (PENDING | REVIEWED | APPROVED | REJECTED)  
}
```

4.9 INVENTORY_TRANSACTIONS

Tracks stock changes

```
InventoryTransaction {  
  id  
  tenant_id  
  product_id  
  type (IN | OUT | ADJUSTMENT)  
  quantity  
  reference_order_id  
  timestamp  
}
```

4.10 NOTIFICATIONS

```
Notification {  
  id  
  tenant_id  
  user_id  
  type (EMAIL | WHATSAPP | SYSTEM)  
  title  
  message  
  status (SENT | FAILED | PENDING)  
}
```

4.11 AUDIT_LOGS (IMMUTABLE)

Very important for enterprise-grade system

```
AuditLog {  
  id  
  tenant_id  
  actor_user_id  
  action  
  entity_type  
  entity_id  
  before_state (JSONB)  
  after_state (JSONB)  
  timestamp  
}
```

5. Relationships Overview

Tenant

- ├── Users
- ├── Customers
- ├── Products
- ├── Orders
 - ├── OrderItems
 - ├── AI Draft Orders
- ├── WhatsApp Messages
- ├── AI Logs
- ├── Inventory Transactions
- ├── Notifications
- └── Audit Logs

6. Indexing Strategy (VERY IMPORTANT)

Required Indexes:

Performance-critical queries:

INDEX on tenant_id (ALL TABLES)

INDEX on phone (customers)

INDEX on status (orders)

INDEX on created_at (orders, messages)

INDEX on product sku

AI performance:

GIN INDEX on JSONB fields:

- product.attributes
 - ai_logs.input_data
 - ai_logs.output_data
-

7. Soft Delete Strategy

Instead of deleting:

deleted_at = timestamp

Rules:

- Soft deleted records are excluded by default queries
 - Only admin can restore
-

8. Row Level Security (Supabase)

Every query MUST enforce:

tenant_id = current_tenant

This ensures:

- No cross-business data leak
 - Secure SaaS isolation
-

9. UUID Strategy

All primary keys:

- ✓ UUID v4
- ✓ Generated in backend (NOT database)

Reason:

- Better distributed scaling
 - Avoid predictable IDs
 - Safer for multi-tenant APIs
-

10. Timestamp Rules

Field	Rule
created_at	set once
updated_at	auto-update on change
deleted_at	only soft delete